KLIPPER · DEBUG ORDER · 1/10

HTTP 200 lied to me. Klipper was already dead.

The debugging order for a green screen with a dead system — learned on a real 3D printer.

10 slides · swipe

Anton Petnitsky · Python / agents / backend

The page loaded. The printer didn't.

Mainsail came up. Nginx answered 200 on every probe. Klippy — the layer that actually moves the motors — was stuck in error.

RP2040 disabled.
ACCEL_AUTOCONFIG blocks
still in printer.cfg.

Two layers, one lie.

INTERFACE

nginx

Mainsail renders
HTTP 200 everywhere
looks fine

PROCESS

klippy

RP2040 MCU
unreachable
error state since boot

The status code only proves the first layer answered.

Anton Petnitsky · Python / agents / backend

A dashboard proves the shell, not the system.

UI checks routes and pages. It cannot see the process inside deciding whether your command does anything.

A green interface
is a report, not a proof.

Anton Petnitsky · Python / agents / backend

Debug order: process before looks.

01 Is the process itself alive and ready?

↓

02 Is its configuration exact and wired correctly?

↓

03 Did the transport between layers actually arrive?

↓

04 Only then: is the user-visible state correct?

Skip a layer and you will fix the wrong thing.

Anton Petnitsky · Python / agents / backend

Ask the process, not the page.

```
$ curl -s 127.0.0.1:7125/printer/objects/query?klippy_state

"state":"ready"   ← good

"state":"error"  ← this was the answer

# the 200 came from nginx, not from klippy
```

Status must come from the component that can also fail.

The root cause chain.

01 Accelerometer disconnected for a reason

↓

02 printer.cfg kept the ACCEL_AUTOCONFIG blocks

↓

03 Startup queried the dead RP2040

↓

04 klippy_state = error, UI still 200

Every hop is visible only from the process layer.

Disable config with the hardware — not just the hardware.

Commenting out the RP2040 blocks (not deleting — config history matters), then restart and re-home:

klippy_state: ready.

Success = the motors move, not the page.

Every dependency has an order. Write it down.

Postgres before the backend. Backend before the proxy. Process before the page. If the order is tribal knowledge, the outage is retraining every on-call.

Timing is part of the contract.

Anton Petnitsky · Python / agents / backend

KLIPPER · KEEP THIS · 10/10

Save this for the next green screen you don't trust.

HTTP 200 is a message from the door, not from the house. When in doubt, ask the innermost component that can tell the truth.

Save for your next incident

Anton Petnitsky · Python / agents / backend